Beyond Chunk-Then-Embed: A Comprehensive Taxonomy and Evaluation of Document Segmentation Strategies for Information Retrieval

No Author Given

No Institute Given

Abstract. Document chunking is a critical preprocessing step in dense retrieval systems, yet the design space of chunking strategies remains poorly understood. Recent work has proposed novel approaches ranging from LLM-guided chunking methods such as DenseX [**?**] and Lumber-Chunker [**?**], to contextualized chunking strategies like Late Chunking [**?**], which generates embeddings before segmentation to preserve contextual information. However, existing methods have been evaluated in isolation across different benchmarks, making direct comparisons difficult.
This paper presents a systematic framework that unifies existing chunking strategies along two key dimensions: (1) the *order* of chunking and embedding operations (pre-embedding vs. contextualized chunking), and (2) the *segmentation methods* (traditional fixed-size, sentence/paragraph-based, semantic boundary-based, and LLM-guided approaches). We reproduce and evaluate representative methods from each category across two distinct retrieval paradigms: single-document needle-in-a-haystack retrieval and multi-document web-scale retrieval. Our comprehensive evaluation reveals that ... Our code and evaluation benchmarks are publicly available at `https://xx`.

Keywords: Chunking · Document Ranking · Re-ranking.

1 Introduction

Dense retrieval systems, which represent queries and documents as low-dimensional vectors, have become a core part of modern information retrieval. They power a wide range of applications, from web search engines to advanced retrieval-augmented generation systems. A key challenge in using these systems is fitting long documents into the fixed-size input windows of embedding models. This requires **document chunking**: the process of dividing documents into smaller, meaningful units for indexing and retrieval. The choice of a chunking strategy is not just a preprocessing step; it is an important decision that greatly affects how well retrieval works. Chunks that are too small may not have enough context to answer a query, while oversized chunks can make it harder to find the relevant information and may be too large for the model to process.

Chunking has developed in two main ways: the method of segmentation and the approach to embedding. Traditionally, systems use a *pre-embedding chunking* approach, where documents are first divided and each resulting chunk is then encoded on its own. Within this approach, segmentation methods range from simple fixed-size windows to more advanced techniques based on linguistic boundaries (e.g., sentences or paragraphs [**?**]), changes in meaning [**?**], and even guidance from large language models (LLMs) [**?**,**?**]. More recently, the introduction of embedding models with longer context windows that support thousands of tokens [**?**,**?**,**?**] has allowed for a new *contextualized chunking* approach. Methods like Late Chunking [**?**] are a good example of this change: they first encode a whole document to create context-aware embeddings and only then divide it, keeping useful information that connects the chunks.

Despite this rapid development, progress has been disorganized because there has been no standard way to evaluate these methods. New chunking strategies are often tested in different and unconnected ways, making it impossible to compare them directly. For example, proposition-based methods were evaluated on question-answering datasets using models with a 512-token limit [**?**]. In contrast, LLM-guided methods like LumberChunker were benchmarked on a single-book retrieval task using private embedding APIs [**?**], while Late Chunking was tested on standard BEIR passage retrieval benchmarks [**?**]. These differences cover embedding models, datasets, and even the type of retrieval task (e.g., searching through many documents vs. a single document). As a result, it is unclear whether the differences in performance come from the actual strengths of the chunking strategies themselves or from these other factors in the experiments.

To address these gaps and provide a clear, evidence-based understanding of document chunking, we conduct a careful study guided by the following research questions:

RQ1 *How do different segmentation methods compare in standard multi-document retrieval?* We evaluate fixed-size, sentence/paragraph-based, semantic boundary-based, and LLM-guided chunking methods using traditional pre-embedding chunking on standard BEIR benchmarks.

RQ2 *Do segmentation methods generalize across retrieval paradigms?* We apply the same segmentation methods to single-document needle-in-a-haystack retrieval, testing whether the relative strengths observed in multi-document settings hold when the retrieval scope is constrained to individual documents.

RQ3 *Can contextualized chunking provide consistent improvements across different configurations?* We apply contextualized chunking to all segmentation methods from RQ1 across both retrieval paradigms from RQ2 to determine whether contextualized chunking provides universal benefits.

RQ4 *What factors determine when contextualized chunking outperforms pre-embedding approaches?* We implement Late Chunking as a representative contextualized method and systematically vary chunk size, overlap ratios, document types, and query characteristics to measure both retrieval effectiveness and computational overhead.

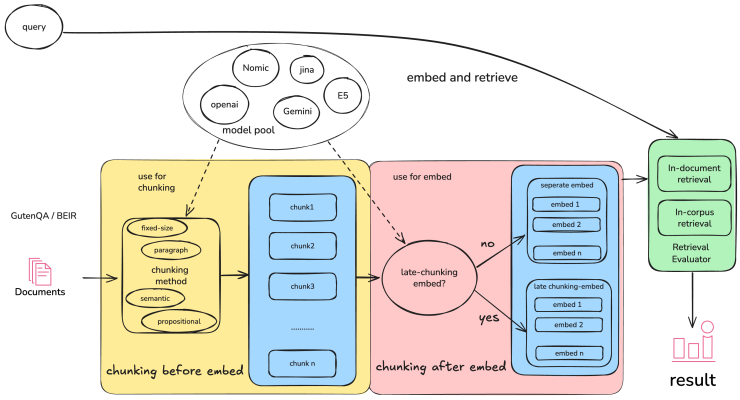

Fig. 1: Unified framework for evaluating document chunking strategies across retrieval paradigms. Documents from BEIR and GutenQA datasets are segmented using one of several methods. Depending on the configuration, embeddings are generated either before chunking (*pre-embedding chunking*) or after full-document encoding (*contextualized chunking*, e.g., Late Chunking). Resulting chunk embeddings are indexed and evaluated under two retrieval settings: (1) *in-document retrieval* (needle-in-a-haystack) and (2) *in-corpus retrieval* (multi-document). The pipeline supports multiple embedding and generative model families, enabling controlled comparison across chunking paradigms and model capacities.

Through investigating these research questions, we (1) establish comprehensive baselines for segmentation methods in multi-document retrieval (**RQ1**), (2) identify which chunking strategies are robust across different retrieval paradigms (**RQ2**), (3) assess whether contextualized chunking provides universal improvements (**RQ3**), (4) uncover the key factors that determine when contextualized chunking justifies its computational overhead (**RQ4**), and (5) release our complete implementation and evaluation framework to facilitate reproducibility and future research.

2 Benchmarking Document Chunking Strategies

Document chunking is the critical step that transforms raw text into the retrievable units used by dense retrieval systems. To systematically compare the wide range of available methods, we organize them along two key dimensions: (1) **segmentation strategy**, which defines how textual boundaries are identified, and (2) **embedding paradigm**, which determines whether chunking occurs before or after the text is encoded into vector representations. This framework provides a clear structure for our experiments and clarifies how different techniques relate to one another.

2.1 Taxonomy of Segmentation Strategies

We classify segmentation strategies into two main families: structure-based heuristics that rely on the document's explicit layout, and semantically-informed methods that use language understanding to find logical divisions.

Structure-Based Heuristics These methods use observable textual features to create chunks. They are widely adopted in practice due to their speed, simplicity, and deterministic output. We evaluate three common heuristic strategies:

- **Paragraphs:** This strategy uses paragraph breaks as chunk boundaries. It respects the document's intended structure at no extra processing cost and is often effective for well-organized texts like news articles.
- **Fixed-Size Windows:** This method divides text into segments of a constant token length (e.g., 512 tokens), with an optional overlap between consecutive chunks. While computationally efficient, it can unnaturally split coherent thoughts or sentences, potentially harming the quality of the resulting embeddings [**?**].
- **Sentences:** This approach forms each chunk using only one sentence. It creates fine-grained segments suitable for tasks like question answering, where answers are often concise. However, it may fail when relevant context is spread across multiple sentences.

Semantically-Informed and LLM-Guided Strategies These advanced methods use models to analyze the text's meaning and identify more natural breakpoints. The aim is to produce chunks that are internally coherent, providing a better foundation for retrieval.

- **Semantic Sentence Splitting:** This technique calculates the semantic similarity between adjacent sentences. When the similarity drops below a certain threshold, indicating a topic shift, a boundary is inserted [**?**]. This adapts to the flow of the document's content.
- **Proposition-Based Chunking:** This method uses an LLM to break down paragraphs into atomic facts or "propositions" [**?**]. Each proposition becomes a chunk, which creates highly specific units ideal for factual lookups. However, this can significantly increase the total number of chunks and is computationally expensive due to its reliance on LLM inference.
- **LumberChunker:** This approach directly instructs an LLM to segment the document by inserting breakpoints between paragraphs where it detects a topic shift [**?**]. While powerful, this method is also slower and more expensive due to its use of LLM inference.

It is important to note that semantically-informed and LLM-guided strategies often build upon structure-based heuristics. For example, semantic sentence splitting relies on an initial sentence segmentation, while both proposition-based chunking and LumberChunker typically operate on paragraph-level text.

2.2 Embedding Paradigms: Pre-Embedding vs. Contextualized Chunking

The second dimension of our framework is the order in which chunking and embedding are performed. The *pre-embedding* paradigm, which has been the traditional approach, segments the document first and then encodes each chunk in isolation. This design is simple and scalable but loses all contextual information across chunks.

In contrast, the *contextualized chunking* paradigm (also known as late chunking) reverses this order. First, a long-context encoder processes the entire document, generating token-level embeddings that are aware of the global context [**?**]. Only then are chunk boundaries applied. The final embedding for each chunk is produced by pooling (e.g., averaging) the embeddings of the tokens it contains. This approach preserves cross-chunk relationships at the cost of higher computational and memory requirements.

2.3 Common Evaluation Paradigms

As noted in the introduction, prior work has used inconsistent settings, making direct comparisons difficult. To ensure a fair evaluation, we test each chunking strategy across the two dominant retrieval paradigms:

- **In-Document Retrieval:** Also known as "needle-in-a-haystack," this task involves finding a specific piece of information within a single long document. It tests a method's ability to precisely locate content.
- **In-Corpus Retrieval:** This is the classic multi-document retrieval task, where the goal is to find the most relevant documents from a large collection, such as the BEIR benchmarks [**?**]. This tests for scalability and effectiveness in a broader search environment.

By systematically evaluating each chunking method across both paradigms with a consistent set of embedding models and controls, our study isolates the impact of the chunking strategy itself, providing a clearer understanding of its true effect on retrieval performance.

3 Experimental Setup

We evaluate each chunking strategy using the unified framework described in Section 2. This section details the specific implementations, datasets, and evaluation procedures used in our experiments.

3.1 Segmentation Methods

To ensure a fair comparison, we use a consistent implementation for each segmentation strategy across all datasets.

For **structure-based heuristics**: the **Paragraphs** strategy defines chunks by newline characters, collapsing consecutive blank lines into a single boundary. The **Fixed-Size** method divides documents into uniform 512-token segments with no overlap. Finally, the **Sentences** approach creates chunks from single sentences, as identified by the spaCy 'en_core_web_sm' model.

For **semantically-informed and LLM-guided strategies**, the **Semantic Sentence** strategy follows the open-source implementation from [**?**], using `jina-embeddings-v2-small-en` to compute the cosine similarity between adjacent sentences and inserting a boundary when the similarity falls below 0.5. For both **Proposition-Based** and **LumberChunker** strategies, we prompt the `Gemini-1.5-Flash` model for chunking document, following previous work from [**?**], to make sure output as deterministic as possible, we set the model generation temperature to 0.0.

3.2 Embedding Models Used

For embedding models used to evaluate the effectiveness of different segmantation strategies and embedding paradigm, we select the following four models:

- **jina-embeddings-v2-small-en**
- **jina-embeddings-v3**
- **nomic-embed-text-v1**
- **multilingual-e5-base**

While the first three models were evaluated in the initial late-chunking method, we also select an additional model multilingual-e5-base to test its robustness, as a nature of late-chunking that has to use average pooling for representation, our selection is constrained by selecting from Mteb leaderboard the highest performing embedding model that is using mean pooling for text representation.

While all of the embedding models are able to handle the *pre-embedding* cases, where each document is firstly chunked and encoded seperatly; during *Contextualized chunking*, it requires to input multiple chunks together to the embedding model, therefore a maximum possible chunk approach is used, that is: each time it will feed the maximum possible number of chunks in a document into the context window of the embedding model; once it exceed, it will create another chunking.

3.3 Datasets and Evaluation

Our evaluation is designed to test the generalizability of chunking strategies across different retrieval tasks and domains.

In-Document Retrieval is evaluated using the **GutenQA** dataset [**?**], which contains over 50 classic literature books. For each book, a set of questions asks for specific information contained within that book. This task tests for precise information localization within a single, long document.

In-Corpus Retrieval is evaluated on a selection of datasets from the **BEIR** benchmark [**?**]: FiQA, ArguAna, SciDocs, TREC-COVID, SciFact, and NFCorpus. We chose these specific datasets based on two criteria. First, they cover a diverse range of retrieval tasks and domains, from finance and scientific articles to biomedical texts. Second, these datasets have a relatively small number of documents, which was a practical consideration to manage the computational cost associated with the LLM-guided chunking methods.

All text is lowercased and tokenized using the specific tokenizer provided with each embedding model.

For **In Document retrieval**, ground-truth is per paragraph, but each chunk may include only a portion of one paragraph, or multiple paragraphs combined together; to distinguish if a retrieved chunk is relevant or not, we match the top n retrieved chunks with the paragraph is this what we do?

On the other hand, for **In-Corpus Retrieval**, ground truth is typically per-document, in this case, we just use the score of the top-ranked chunk of the document to represent the relevance of the document. As specified by previous paper by [**?**], this is the most effective way ...

For evaluation metrics, we use Following standard practices for these datasets, we report nDCG@10 (normalized Discounted Cumulative Gain at rank 10) for all BEIR tasks and DCG@10 for GutenQA. The formula for DCG is:

$$\text{DCG@10} = \sum_{i=1}^{10} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}$$

where rel_i is the relevance score of the result at rank i. The nDCG@10 metric normalizes this score by the best possible DCG@10 score, producing a value between 0 and 1. All metrics are calculated using the official BEIR evaluation toolkit.

4 Result

4.1 RQ1: Impact of Chunking Strategies in Multi-Document Retrieval

To establish baseline effectiveness characteristics of different chunking strategies, we evaluated six chunkers across seven BEIR datasets using four state-of-the-art embedding models. Table 1 presents the results using traditional pre-embedding chunking, focusing on the corpus-level retrieval columns (fiqa through scidocs).

No single chunking strategy dominates across all scenarios. The effectiveness of chunkers varies significantly depending on the embedding model and dataset characteristics. Across the six corpus-level retrieval tasks, paragraph-based chunking achieves the highest average effectiveness for three out of four models (Jina-v2: 0.4303, Jina-v3: 0.4948, E5-Large: 0.4997), while fixed-size chunking performs best with Nomic embeddings (0.4828). This suggests that the optimal strategy depends on how different embedding models encode semantic boundaries.

LLM-guided strategies underperform in multi-document settings. Contrary to expectations, sophisticated LLM-guided approaches (LumberChunker and proposition-level chunking) do not outperform simpler chunkers. Proposition-based chunking consistently yields the lowest average scores across all models (ranging from 0.3487 to 0.3916), while LumberChunker ranks fourth or fifth in average corpus effectiveness. For instance, with Jina-v3, paragraph chunking outperforms LumberChunker by 11% (0.4948 vs 0.4463) and proposition-level chunking by 26% (0.4948 vs 0.3916).

Dataset characteristics influence chunking effectiveness. Effectiveness patterns vary across different corpus types. For domain-specific datasets like SciFact and TREC-COVID, sentence-based chunking often performs competitively or best (e.g., SciFact with Jina-v2: 0.6548, TREC-COVID with Nomic: 0.7859 for fixed-size). In contrast, for argumentative text (ArguAna), paragraph and fixed-size chunking consistently outperform more sophisticated strategies, suggesting that natural discourse boundaries are crucial for certain retrieval tasks.

Model architecture affects chunking sensitivity. The variance in effectiveness across chunkers differs substantially between models. Jina-v2 shows the highest sensitivity to chunking choice with a 23% effectiveness gap between best and worst strategies (0.4303 vs 0.3487), while E5-Large shows more robustness with only a 11% gap (0.4997 vs 0.3883). This indicates that some embedding models are inherently better at handling suboptimal chunk boundaries.

Overall, our results reveal that simple chunking strategies are most effective for multi-document retrieval. Paragraph-based chunking emerges as the most reliable choice, achieving the highest average effectiveness in three of four models and consistently ranking in the top-2 across individual datasets. Fixed-size chunking offers comparable effectiveness (within 2% of paragraph chunking on average) with computational advantages. Surprisingly, LLM-guided strategies significantly underperform despite their sophistication: proposition-level chunking averages 15-20% lower effectiveness than paragraph chunking across all models. This answers RQ1 by demonstrating that for standard multi-document retrieval, simpler chunkers that respect natural document boundaries or use consistent sizes provide superior effectiveness and robustness compared to semantic-aware alternatives.

5 RQ2: Do Chunking Strategies Generalize Across Retrieval Paradigms?

We contrast effectiveness on the single-document GutenQA benchmark with the multi-document BEIR tasks to evaluate cross-paradigm robustness. Table 1 shows that the chunkers excelling on GutenQA are precisely those that falter on BEIR. Proposition-level chunking delivers the highest GutenQA scores for three of the four encoders (e.g., 0.5556 DCG@10 with Jina-v2 and 0.5530 with E5-Large), yet it yields the lowest average corpus effectiveness in every case, trailing paragraph chunking by 18% relative for Jina-v2 (0.3487 vs. 0.4303) and

Table 1: Effectiveness comparison across different models, chunking strategies, and datasets using DCG@10 for GutenQA and nDCG@10 for other datasets (regular encoder)

		document	***corpus***						
		GutenQA	fiqa	nfcorpus	scifact	trec-covid	arguana	scidocs	Avg
Jina-v2	Paragraph	0.3893	**0.3347**	**0.3042**	0.6389	0.6518	0.4682	0.1843	**0.4303**
	Sentence	0.3261	0.3096	0.2847	**0.6548**	**0.6674**	0.4675	0.1732	0.4262
	Fixed-Size	0.3889	0.3339	0.2604	0.6350	0.6437	**0.4706**	**0.1853**	0.4215
	Semantic	0.3858	0.3030	0.2746	0.6445	0.6618	0.4022	0.1760	0.4104
	Lumber	0.4782	0.2754	0.2887	0.6540	0.6456	0.3906	0.1721	0.4044
	Proposition	**0.5556**	0.2186	0.2995	0.6331	0.4292	0.3681	0.1436	0.3487
	Avg	0.4207	0.2959	0.2854	0.6434	0.6166	0.4279	0.1724	0.4069
Jina-v3	Paragraph	0.4574	**0.4747**	**0.3663**	0.7247	**0.7721**	**0.4327**	0.1984	**0.4948**
	Sentence	0.3877	0.4333	0.3566	0.7163	0.7135	0.4256	0.1888	0.4723
	Fixed-Size	0.4827	0.4728	0.3596	**0.7262**	0.7557	0.4325	**0.1985**	0.4909
	Semantic	0.4105	0.4396	0.3596	0.7103	0.7474	0.3894	0.1890	0.4726
	Lumber	**0.5693**	0.4028	0.3534	0.6845	0.6900	0.3671	0.1797	0.4463
	Proposition	0.5484	0.2858	0.3579	0.6690	0.5756	0.3044	0.1569	0.3916
	Avg	0.4760	0.4182	0.3589	0.7052	0.7091	0.3920	0.1852	0.4614
Nomic	Paragraph	0.4893	0.3802	0.3509	0.7047	0.7652	0.4904	0.1808	0.4787
	Sentence	0.4146	0.3620	0.3502	**0.7150**	0.7472	**0.5084**	0.1732	0.4760
	Fixed-Size	0.5104	**0.3839**	0.3492	0.7050	**0.7859**	0.4915	**0.1812**	**0.4828**
	Semantic	0.4274	0.3562	0.3506	0.7040	0.7431	0.4434	0.1707	0.4613
	Lumber	**0.6048**	0.3347	**0.3513**	0.7093	0.7193	0.4315	0.1673	0.4522
	Proposition	0.5827	0.2575	0.3358	0.6853	0.5580	0.3693	0.1441	0.3916
	Avg	0.5049	0.3457	0.3480	0.7039	0.7198	0.4557	0.1696	0.4571
E5-Large	Paragraph	0.4663	**0.4617**	0.3566	**0.7129**	0.6634	**0.5999**	0.2038	**0.4997**
	Sentence	0.4063	0.4315	0.3583	0.6873	0.6712	0.5646	0.1836	0.4828
	Fixed-Size	0.4862	0.4599	0.3495	0.7078	0.6590	0.5987	**0.2058**	0.4968
	Semantic	0.3186	0.4173	0.3583	0.6892	0.6784	0.5540	0.1888	0.4810
	Lumber	0.5234	0.3739	0.3531	0.6748	**0.6931**	0.5233	0.1792	0.4663
	Proposition	**0.5530**	0.2522	**0.3602**	0.6851	0.5611	0.3163	0.1551	0.3883
	Avg	0.4590	0.3994	0.3560	0.6928	0.6544	0.5261	0.1861	0.4691

by 22% for E5-Large (0.3883 vs. 0.4997). LLM-guided LumberChunker exhibits a similar pattern: it leads the GutenQA column for Jina-v3 (0.5693) but drops to fourth place on BEIR (0.4463 average). In contrast, simple heuristics, namely paragraph and fixed-size chunking, rarely win on GutenQA but consistently occupy the top two ranks on the multi-document tasks.

These trends persist when we switch to contextualized encoders in Table 2. Late chunking boosts proposition-based scores on GutenQA (0.5280 with Jina-v2, 0.5117 with E5-Large), yet its BEIR averages remain far below those of the heuristic baselines (0.4049 and 0.4030, respectively). Sentence blocks and fixed-size windows, while modest on document retrieval, retain their edge on BEIR after contextualization (e.g., 0.4400 for Jina-v2 sentence blocks, 0.4770 for Nomic sentence blocks). The lack of a single method that performs strongly in both paradigms demonstrates that chunkers tuned for precise passage localization do not generalize to open-domain retrieval, whereas coarse structural heuristics are more transferable even if they sacrifice some in-document fidelity.

6 RQ3: Effectiveness of Contextualized Chunking

We next examine whether contextualized chunking (late encoder) provides consistent gains over pre-embedding baselines. Table 3 reports relative changes when switching from the regular encoder to the late encoder. Improvements are highly localized: sentence blocks with Jina-v2 gain 6.5% on nfcorpus and 9.6% on FiQA, while losing 5.4% DCG@10 on GutenQA. Fixed-size windows also benefit slightly on nfcorpus (+15.3%) yet deteriorate on the document task (-2.3%). Paragraph chunking, which is the strongest BEIR baseline, rarely improves; its average change across corpora is within +/-0.5% for three encoders, and it suffers double-digit drops on GutenQA (e.g., -10.7% for Jina-v2, -39.4% for Nomic).

LLM-driven methods experience the largest swings. Proposition-level chunking enjoys dramatic relative gains on argumentative corpora when paired with contextualized embeddings (e.g., +42.2% nDCG on ArguAna with Jina-v2 and +59.8% with E5-Large), but those gains coincide with severe regressions on cleaner domains (-25.9% on TREC-COVID for E5-Large, -19.3% for Nomic). LumberChunker similarly oscillates: contextualization boosts ArguAna by up to +27.8% for Jina-v2 yet erodes FiQA by as much as -29.1%.

Overall, the late encoder delivers at most marginal average gains across BEIR (no more than +2.4 percentage points) while incurring substantial cost on the single-document task (up to -62.5% for paragraph chunking with Nomic). These results indicate that contextualized chunking is not a free upgrade: it must be targeted at specific chunker-model-dataset combinations where the additional context outweighs the noise introduced by pooling across long sequences.

Table 2: Effectiveness comparison across different models, chunking strategies, and datasets using DCG@10 for GutenQA and nDCG@10 for other datasets (late encoder)

		document	***corpus***						
		GutenQA	fiqa	nfcorpus	scifact	trec-covid	arguana	scidocs	Avg
Jina-v2	Paragraph	0.3476	0.3346	0.3041	0.6397	0.6618	0.4666	0.1854	0.4320
	Sentence	0.3084	**0.3392**	0.3033	0.6583	**0.6791**	0.4764	0.1839	**0.4400**
	Fixed-Size	0.3799	0.3351	0.3003	0.6448	0.6649	0.4685	**0.1858**	0.4332
	Semantic	0.4041	0.2195	0.2900	0.6489	0.6076	0.4715	0.1836	0.4035
	Lumber	0.4290	0.2223	0.3079	0.6527	0.6516	0.4759	0.1855	0.4160
	Proposition	**0.5280**	0.3042	**0.3210**	**0.6632**	0.4884	**0.4869**	0.1655	0.4049
	Avg	0.3995	0.2925	0.3044	0.6513	0.6256	0.4743	0.1816	0.4216
Jina-v3	Paragraph	0.2816	0.4306	0.3442	0.6826	0.6921	**0.4732**	0.2034	0.4710
	Sentence	0.2524	**0.4355**	0.3443	0.6841	**0.7046**	0.4605	0.2011	**0.4717**
	Fixed-Size	0.3411	0.4308	0.3442	0.6856	0.6881	0.4732	**0.2036**	0.4709
	Semantic	0.3356	0.2895	0.3474	0.6880	0.6527	0.4729	0.2019	0.4421
	Lumber	0.3789	0.2857	0.3506	**0.6947**	0.6826	0.4691	0.2020	0.4474
	Proposition	**0.4984**	0.3670	**0.3551**	0.6937	0.5229	0.4328	0.1883	0.4266
	Avg	0.3480	0.3732	0.3476	0.6881	0.6571	0.4636	0.2001	0.4550
Nomic	Paragraph	0.1839	**0.3801**	0.3508	0.7057	0.7361	0.4936	**0.1808**	0.4745
	Sentence	0.1557	0.3777	0.3536	0.7129	**0.7517**	0.4883	0.1778	**0.4770**
	Fixed-Size	0.2127	0.3796	0.3515	0.7058	0.7316	**0.4941**	0.1806	0.4738
	Semantic	0.2435	0.2804	0.3535	0.7036	0.7052	0.4890	0.1786	0.4517
	Lumber	0.2395	0.2757	**0.3604**	**0.7191**	0.6790	0.4823	0.1796	0.4494
	Proposition	**0.4213**	0.3145	0.3515	0.7124	0.4502	0.4773	0.1530	0.4098
	Avg	0.2428	0.3347	0.3535	0.7099	0.6756	0.4874	0.1751	0.4560
E5-Large	Paragraph	0.4223	**0.4613**	**0.3576**	**0.7152**	**0.6630**	**0.5994**	**0.2064**	**0.5005**
	Sentence	0.3621	0.4608	0.3553	0.7030	0.6479	0.5945	0.2061	0.4946
	Fixed-Size	0.4863	0.4601	0.3510	0.7095	0.6615	0.5978	0.2063	0.4977
	Semantic	0.3259	0.3227	0.3549	0.7006	0.6430	0.5923	0.2055	0.4698
	Lumber	**0.5245**	0.3015	0.3518	0.6976	0.6137	0.5922	0.2050	0.4603
	Proposition	0.5117	0.3306	0.3405	0.6427	0.4160	0.5056	0.1826	0.4030
	Avg	0.4388	0.3895	0.3518	0.6948	0.6075	0.5803	0.2020	0.4710

Table 3: Late-encoder effectiveness relative to the regular encoder (baseline). Values show percentage change (light green=improvement, light red=degradation)

		document	***corpus***						
		GutenQA	fiqa	nfcorpus	scifact	trec-covid	arguana	scidocs	Avg
Jina-v2	Paragraph	-10.72	-0.02	-0.03	+0.12	+1.53	-0.33	+0.61	+0.31
	Sentence	-5.43	+9.57	+6.52	+0.52	+1.76	+1.89	+6.17	+4.40
	Fixed-Size	-2.32	+0.35	**+15.32**	+1.55	+3.30	-0.44	+0.27	+3.39
	Semantic	**+4.75**	-27.57	+5.59	+0.69	-8.19	+17.22	+4.31	-1.32
	Lumber	-10.30	-19.29	+6.66	-0.19	+0.93	+21.85	+7.80	+2.96
	Proposition	-4.98	**+39.17**	+7.18	**+4.75**	**+13.79**	**+32.26**	**+15.29**	**+18.74**
	Avg	-4.83	+0.37	+6.87	+1.24	+2.19	+12.07	+5.74	+4.75
Jina-v3	Paragraph	-38.43	-9.28	-6.02	-5.81	-10.37	+9.37	+2.52	-3.27
	Sentence	-34.89	+0.51	-3.43	-4.49	-1.25	+8.20	+6.55	+1.01
	Fixed-Size	-29.35	-8.88	-4.27	-5.58	-8.95	+9.40	+2.57	-2.62
	Semantic	-18.24	-34.15	-3.41	-3.14	-12.67	+21.43	+6.85	-4.18
	Lumber	-33.46	-29.08	-0.78	+1.48	-1.08	+27.78	+12.40	+1.79
	Proposition	-9.12	**+28.41**	-0.78	**+3.70**	-9.16	**+42.20**	**+20.01**	**+14.06**
	Avg	-27.25	-8.75	-3.11	-2.31	-7.25	+19.73	+8.48	+1.13
Nomic	Paragraph	-62.41	-0.03	-0.03	+0.14	-3.80	+0.65	+0.02	-0.51
	Sentence	-62.46	+4.33	+0.98	-0.28	**+0.60**	-3.96	+2.66	+0.72
	Fixed-Size	-58.33	-1.10	+0.63	+0.11	-6.91	+0.51	-0.36	-1.19
	Semantic	-43.04	-21.28	+0.82	-0.04	-5.09	+10.29	+4.62	-1.78
	Lumber	-60.40	-17.64	+2.61	+1.39	-5.59	+11.78	**+7.35**	-0.02
	Proposition	-27.70	**+22.15**	**+4.68**	**+3.95**	-19.32	**+29.26**	+6.22	**+7.82**
	Avg	-52.39	-2.26	+1.62	+0.88	-6.69	+8.09	+3.42	+0.84
E5-Large	Paragraph	-9.43	-0.09	+0.27	+0.32	-0.05	-0.08	+1.27	+0.27
	Sentence	-10.89	+6.80	-0.84	+2.29	-3.48	+5.29	+12.28	+3.72
	Fixed-Size	+0.01	+0.05	**+0.42**	+0.25	**+0.38**	-0.14	+0.25	+0.20
	Semantic	**+2.29**	-22.66	-0.96	+1.66	-5.22	+6.92	+8.83	-1.90
	Lumber	+0.20	-19.37	-0.39	**+3.37**	-11.46	+13.18	+14.38	-0.05
	Proposition	-7.47	**+31.09**	-5.47	-6.18	-25.85	**+59.84**	**+17.77**	**+11.87**
	Avg	-4.22	-0.69	-1.16	+0.28	-7.61	+14.17	+9.13	+2.35

7 RQ4: Other Factors

Acknowledgment

References